

Objects:-

An Object is a collections of related data and functionality.

Objects stores Property value pairs/ Key value pairs.

Example :-

```
const person1 = {
  name : "Vaishvik",    // name is a property and "Vaishvik" is a value
  age : 24,
  city : "Shimla"
}
```

```
// name , age, city are all properties(keys)
// "vaishvik", 24, "Shimla" are all values.
```

-- Values:-

Values can be :-

String

Number

Boolean

Array

Function

Another Object(Nested Object)

Null

Undefined

Example :-

```
const car1 = {
  brand : "Toyota",
  model : "Innova",
  year : 2025,
  available : true,
  color : ["Blue", "Red", "White"],
  address : {
    state : "Himachal Pradesh",
    city : "Shimla"
  }
};
```

```
console.log(car1.address.city);
```

-- How to add a new property to an Object

Example:-

```
const user1 = {
```

```
    name : "Aman"
  }
```

**** Adding a new value with dot notation:-**

```
user1.age = 22;
console.log(user1);
```

output :-

```
{name : "Aman" , age : 22}
```

**** Adding a new value using bracket notation:-**

Example:-

```
const product = {
  name : "Iphone 17",
  price : 900000,
  brand : "Apple"
}
```

```
product["delivery-time"] = "2 days";
console.log(product);
```

Output :-

```
{
  name : "Iphone 17",
  price : 900000,
  brand : "Apple",
  "deliver-time" : "2 days"
}
```

How to delete an Existing Property:-

use delete keyword and property name to delete it from the object.

Example :-

```
const product = {
  name : "Iphone 17",
  price : 900000,
  brand : "Apple",
  "delivery-time" : "2 days"
}
```

```
// delete product["brand"];
delete product.brand; // to delete a property we will use delete keyword
```

```
console.log(product);
```

Output:-

```
{
  name : "Iphone 17",
  price : 900000,
  "delivery-time" : "2 days"
}
```

How to update an existing property:-

```
const product = {
  name : "Iphone 17",
  price : 900000,
  brand : "Apple",
  "delivery-time" : "2 days"
}
```

```
product.price = 70000;
console.log(product);
```

Output :-

```
{
  name : "Iphone 17",
  price : 700000,
  brand : "Apple",
  "delivery-time" : "2 days"
}
```

Nested Objects:-

Objects can contain other objects. Those objects are known as nested objects.

```
const user = {
  name : "Vaisvrik",
  address : {
    city : "Shimla",
    town : "Hatkoti",
    state : "Himachal"
  }
}
```

address is a nested object.

-- How to access properties of a nested object

```
let city = user.address.city;
console.log(city);
```

Methods Inside Objects:-

```
const user1 = {
  name : "vaishvik",
  greet : function functionName(){
    console.log("Hello, how are you?")
  }
}

const user2 = {
  name : "John",
  greet : function functionName(){
    console.log("Howdy, How are you doing?");
  }
}
```

(Short hand method) You can Write the above code as :-

```
const user1 = {
  name : "vaishvik",
  greet(){ // short hand method
    console.log("Hello, how are you?")
  }
}

const user2 = {
  name : "John",
  greet(){ // short hand method
    console.log("Howdy, How are you doing?");
  }
}

user1.greet();
user2.greet();
```

The "this" keyword inside Methods:-
Inside objects, this refers to the current object

Example :-

```

const user1 = {
  name : "vaishvik",
  greet(){ // short hand method
    console.log(`Hello ${this.name}`)
  }
}

const user2 = {
  name : "John",
  greet(){
    console.log(`Hello ${this.name}`);
  }
}

user1.greet(); //Hello vaishvik
user2.greet(); // Hello John

```

 Inbuilt objects :-

- 1) JSON
- 2) localStorage

1) JSON :- JavaScript object Notation.

It is a lightweight text format used to store and exchange data between
 frontend and backend
 browser and server
 application and database

JSON supports:-

String, Number, Boolean, null, object, array

JSON doesn't support :-

function, undefined, comments, Date objects directly

js object

```

{
  name : "vashi",
  greet : function fun(){
    console.log("Hello");
  }
}

```

this converted into JSON will look like

JSON

```

{

```

```
        "name" : "vashi"
    }
}
```

Example of JSON:-

JavaScript Object:-

```
{
  name : "Vaishvik",
  city : "Shimla",
  skills : {
    skill1 : "Java",
    skill2 : "JavaScript",
    skill3 : "React"
  }
}
```

JSON :-

```
{
  "name" : "Vaishvik",
  "city" : "Shimla",
  "skills" : {
    "skill1" : "Java",
    "skill2" : "JavaScript",
    "skill3" : "React"
  }
}
```

JSON methods:-

1) stringify() :-

JSON.stringify() is used to convert JavaScript objects to JSON string.

Syntax:-

JSON.stringify(JS Object)

Example:-

```
let person1 = {
  name : "Vaishvik",
  city : "Shimla",
  skills : {
    skill1 : "Java",
    skill2 : "JavaScript",
    skill3 : "React"
  }
}
```

```
};  
  
const jsonData = JSON.stringify(person1);  
console.log(jsonData);
```

2) parse():-

Converts JSON string back into JavaScript object

Example:-

```
let person1 = {  
  name : "Vaishvik",  
  city : "Shimla",  
  age : 24,  
  skills : {  
    skill1 : "Java",  
    skill2 : "JavaScript",  
    skill3 : "React"  
  }  
};  
  
const jsonData = JSON.stringify(person1); // converted object into JSON string  
  
const person1copy = JSON.parse(jsonData); // converted JSON string back into  
JavaScript object  
  
console.log(person1copy); // person1copy stores an object.
```

localStorage:-

localStorage is a browser feature that stores data permanently inside the browser.

- setItem():-

setItem() is used to store data in localStorage

syntax :-

```
localStorage.setItem("name", "vaishvik");
```

- getItem():-

getItem() is used to get data from localStorage

Syntax :-

```
localStorage.getItem("name"); // returns :- vaishvik
```

- removeItem()
removeItem() removes key value from the localStorage
removes only one item
Syntax:-
localStorage.removeItem("name"); // removes name from the localStorage

- clear()
clear() removes everything from localStorage

syntax:-
localStorage.clear();

Destructuring in JavaScript:-

Destructuring means extracting values from arrays or objects into variables.

Example of object destructuring :-

```
const person = {  
  name : "Khudeja",  
  age : 21,  
  city : "Pune",  
  skills : ["Java","DB","Aws"]  
};
```

```
// let name = person.name;  
// let age = person.age;
```

```
let {name, age, state} = person; // This is object destructuring
```

```
console.log(name); // Khudeja  
console.log(age); // 21  
console.log(state); // no property named state inside person object, so state will  
return undefined.
```

Reference copy - Shallow copy - Deep copy

- Primitive and non-primitive values:-

- Primitive values:- Primitive values are copied by value.
- number
- string
- Boolean
- null
- undefined
- bigint
- symbol

Non primitive values:-

- object
- array

Reference copy :-

Objects and arrays are copied by reference, not by actual data.
That means variables store the memory address of the object.

Example :-

```
let person1 = {  
  name : "sai",  
  age : 23  
}
```

```
let person2 = person1;  
person2.name = "Rohan";
```

```
console.log(person1.name); //Rohan  
console.log(person2.name); // Rohan
```

Shallow copy:-

A shallow copy creates a new outer object, but nested objects/arrays are still shared.

Shallow copy with spread operator:-

```
const product1 = {  
  name : "iphone 14",  
  price : 34000,  
  specifications : {  
    ram : 6,  
    rom : 64,  
    processor : "A15"  
  }  
}
```

```
const product2 = {...product1}; // shallow copy with spread operator
const product3 = product1; // reference copy
```

```
product1.name = "Iphone 18";
product2.name = "Galaxy s26";
product2.price = 70000;
product2.specifications.processor = "Snapdragon 800";
```

```
console.log("Product 1");
console.log(product1.name); // iPhone 18
console.log(product1.specifications.processor); // Snapdragon 800
```

```
console.log("Product 2");
console.log(product2.name); // Galaxy s26
console.log(product2.specifications.processor); // Snapdragon 800
```

```
console.log("Product 3");
console.log(product3); // iPhone 18
console.log(product3.specifications.processor); // Snapdragon 800
```

Spread operator:- ({...objectName}/{...arrayName})

Spread operator creates a new array/object, and it copies all the top level properties from the old object into the new object, but nested non-primitive values are shared.

```
let user1 = {
  name : "ron"
}
```

```
let user2 = {...user1};
```

#Deep Copy:-

A deep copy creates completely independent copies of outer object, nested object, nested arrays.

1) structuredClone():-

Example :-

```
const product1 = {
  name : "iphone 14",
```

```

    price : 34000,
    specifications : {
      ram : 6,
      rom : 64,
      processor : "A15"
    },
    fun(){
      console.log("This is a phone");
    }
  }
}

```

```

const product2 = structuredClone(product1); // copy an object with the help of
structuredClone()

```

```

product2.name = "S26";
product2.price = 50000;
product2.specifications.processor = "Snapdragon 800";
product2.specifications.ram = 8;

product2.fun();

console.log(product1);
console.log(product2);

```

2) deep copy with stringify() :-

```

const product1 = {
  name : "iphone 14",
  price : 34000,
  specifications : {
    ram : 6,
    rom : 64,
    processor : "A15"
  },
  fun(){
    console.log("This is a phone");
  }
};

const product2 = JSON.parse(JSON.stringify(product1));
product2.specifications.processor = "Snapdragon 400";

console.log(product1.specifications.processor); // "A15"
console.log(product2.specifications.processor); // "Snapdragon 400"

```

